KnowFeat: Knowledge-Guided Feature Engineering via LLM Agents

Chengsong You[1], Wangyue Li[1], Weiqiao Que[1], Qizhou Chen[1],
Kunyan Wu[2], Wei Deng[3], Feng Zhu[4], Xiaofeng He[1,*]
[1]East China Normal University, Shanghai, China
{51275901122, 52285901017}@stu.ecnu.edu.cn, weiqiaoque@gmail.com,
582476065@qq.com, hexf@cs.ecnu.edu.cn
[2]Tsinghua University, Beijing, China
wky24@mails.tsinghua.edu.cn
[3]Southwestern University of Finance and Economics, Chengdu, China
dengwei@swufe.edu.cn
[4]Digital Currency Department, Postal Savings Bank of China, Beijing, China
Jason_bank@126.com

Abstract—Automated feature engineering with large language models (LLMs) can produce semantically meaningful features for tabular data, yet existing methods lack structured domain knowledge, rigorous verification, and explainable provenance. We propose KnowFeat, a knowledge-guided feature engineering framework that organizes domain knowledge into five types—schema metadata, regulatory indicators, detection rules, expert opinions, and court document evidence—and injects them as structured context into an LLM agent. A three-stage verification pipeline filters candidates through code execution, statistical quality checks, and model effectiveness evaluation. Every accepted feature carries a provenance record tracing its design to specific knowledge assets. Under a strict held-out protocol that eliminates feature-selection leakage, KnowFeat ranks first (avg. rank 2.3) across twelve public benchmarks among seven methods (one-sided Wilcoxon p=0.017), with a peak gain of +11.6 pp AUC on a telecom churn dataset. On a real-world Bitcoin anti-money laundering (AML) dataset (Elliptic) and a synthetic digital currency AML benchmark (SimECNY), KnowFeat maintains competitive detection performance with full provenance traceability.

I. Introduction

Feature engineering is among the most impactful steps in building machine learning models for tabular data [1]. Good features improve accuracy and make models easier to interpret—but designing them by hand requires domain expertise that is scarce and does not transfer between applications.

Traditional automated feature engineering (AutoFE) methods such as OpenFE [2] and AutoFeat [3] apply systematic transformations (mathematical operations, aggregations, and feature crosses) to existing columns, then select the most predictive ones. These methods are purely data-driven: they explore a predefined transformation space without understanding what the features mean. In regulated domains such as finance and healthcare, stakeholders must know what a feature represents before deploying it—semantic meaning is not optional.

*Corresponding author.

The emergence of large language models (LLMs) has opened a new paradigm for feature engineering. Methods such as CAAFE [4], FeatLLM [5], and LLM-FE [6] use LLMs to generate feature transformation code from dataset descriptions, leveraging the model's broad world knowledge to propose semantically meaningful features. These methods show gains on general benchmarks but were not designed for regulated, domain-specific settings. Three limitations stand out:

(1) Lack of structured domain knowledge. LLM-based methods rely on the model's parametric knowledge, which is often outdated or absent for specialized domains such as anti-money laundering (AML) or clinical risk scoring. Rich external knowledge sources exist—regulatory indicators, detection rules, expert opinions, court evidence—but no existing method organizes them into a structured taxonomy for systematic injection into feature generation.

(2) Insufficient feature verification. Even when useful candidate features are generated, verification is weak. CAAFE relies on a logistic regression proxy; LLM-FE uses downstream accuracy without statistical filtering; FeatLLM performs no model-based validation at all. Unverified features—syntactically valid but statistically redundant or model-harmful—enter the training data unchecked.

(3) Limited explainability. In regulated industries, model features must be both explainable and auditable. Decision-makers need to know why a feature was created and what evidence supports it. No existing method provides structured provenance linking each feature to specific knowledge assets.

We propose KnowFeat (Knowledge-guided Feature engineering), a framework that tackles all three:

- Multi-Type Knowledge Injection. Domain knowledge is organized into five types of increasing specificity (schema metadata, risk indicators, detection rules,

expert opinions, and court document evidence) and injected as structured context into the LLM agent.
- Three-Stage Verification Pipeline. Each candidate passes through L1 (code execution sandbox), L2 (statistical quality with noise-calibrated thresholds), and L3 (model effectiveness via XGBoost across 5 seeds). Early stages are cheap; expensive evaluation runs only on candidates that survive.
- Explainable Provenance. Each accepted feature carries a provenance card recording the knowledge assets, reasoning chain, computation steps, and verification outputs.

The source code[1] and datasets[2] are publicly accessible.

We evaluate KnowFeat on twelve public tabular datasets, a synthetic AML benchmark (SimECNY, 200K transactions), and a real-world Bitcoin AML dataset (Elliptic, 46K labeled transactions). All experiments use a strict held-out protocol that eliminates the feature-selection leakage present in prior work. Key findings:

- On public datasets, KnowFeat ranks first (avg. rank 2.3) among seven methods under leakage-free evaluation, with a significant one-sided Wilcoxon test (p=0.017). The largest gain is +11.6 pp AUC on a telecom churn dataset.
- On both AML datasets, KnowFeat's verified features remain competitive with strong baselines; on SimECNY, FeatLLM's 1,124-feature set lowers F1, showing that volume without verification hurts.
- Every accepted feature carries a provenance record linking it to specific knowledge assets, enabling audit in regulated domains.

II. Related Work

A. Traditional Automated Feature Engineering

Traditional AutoFE methods search a syntactic transformation space. Deep Feature Synthesis [7] applies aggregation primitives along relational paths; ExploreKit [8] enumerates operators and ranks candidates via meta-learning; AutoFeat [3] uses polynomial expansion with L1 selection; and OpenFE [2] frames feature generation as an operator-operand search with LightGBM-based importance ranking. End-to-end AutoML systems such as Auto-sklearn [9] and FLAML [10] include feature preprocessing but do not generate semantically new features. These methods apply predefined mathematical operators without semantic understanding: age × income is treated identically to age × zip_code, even though only the former has financial meaning. They cannot generate domain-specific features (e.g., "transaction velocity" in anomaly detection) that require conceptual understanding [11].

[1]https://github.com/youchengsong/large-llm-create-feature

[2]https://huggingface.co/datasets/anonymous-7219/knowfeat

B. LLM-Based Feature Engineering

Large language models [12], [13] have enabled a shift from syntactic to semantic feature generation. CAAFE [4] iteratively prompts an LLM with dataset descriptions and evaluation feedback to generate Python code for new features, using logistic regression as a proxy evaluator. FeatLLM [5] takes a two-pass approach: first extracting natural language rules from the LLM, then converting these rules into binary indicator features. LLM-FE [6] frames feature engineering as an evolutionary optimization problem, using LLMs to iteratively generate and refine features through structured prompts with model performance feedback. OCTree [14] guides LLM-based feature generation with decision tree reasoning, using tree structure feedback to steer the generation process. In the financial domain, RiskTagger [15] uses an LLM agent with regulatory knowledge to annotate Web3 crypto laundering behaviors, demonstrating that domain knowledge can enhance LLM-based data annotation.

These methods vary in their knowledge use and verification. LLM-FE incorporates limited domain context (dataset descriptions), but none organizes knowledge into a structured taxonomy for systematic injection. Verification also varies: CAAFE uses a weak proxy model, LLM-FE uses downstream model accuracy, OCTree uses decision tree feedback, and FeatLLM skips model-based validation entirely. None combines code execution, statistical filtering, and model evaluation in a single pipeline, nor traces each feature back to specific knowledge assets.

Tool-augmented LLM agents [16] have been applied to code generation [17], software engineering [18], data analysis [19], and scientific discovery [20]. In data science, AIDE [21] and DS-Agent [22] automate end-to-end pipelines but do not address knowledge-grounded feature generation. KnowFeat builds on the tool-use paradigm by equipping an LLM agent with specialized tools for data exploration, feature generation with provenance, and multi-stage verification.

Table I summarizes the differences. KnowFeat is, to our knowledge, the first LLM-based feature engineering method that combines structured domain knowledge injection, multi-stage verification, and feature provenance tracking.

C. Domain Knowledge Integration in Machine Learning

Knowledge-infused learning [23] embeds external knowledge graphs into neural architectures, and transfer learning [24] transfers knowledge through pretrained representations, but both target unstructured data rather than tabular features. In financial AML, domain experts manually encode risk indicators, transaction structuring patterns, and network centrality measures as features [25]. Regulatory bodies publish red-flag typologies (e.g., FATF guidelines [26]), and institutions maintain internal detection rules from compliance experience and legal proceed-

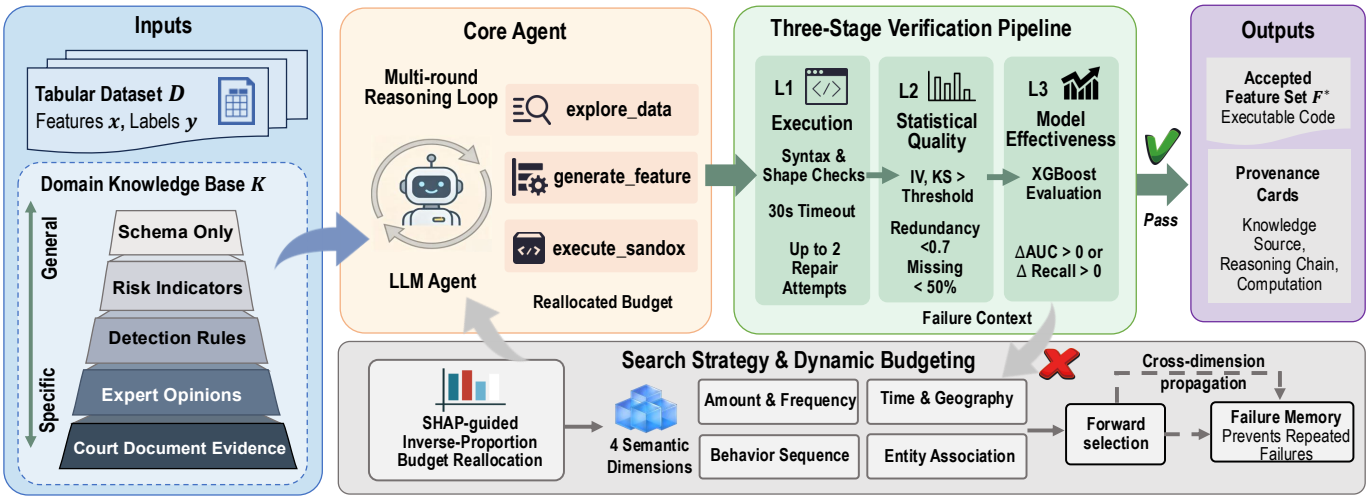


Fig. 1. Overview of the KnowFeat framework. The LLM agent generates features guided by structured domain knowledge. Each candidate is verified through a three-stage pipeline (L1–L3). SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

TABLE I
Comparison of automated feature engineering methods. ✓: fully supported; ○: partially supported; blank: not supported; –: not applicable.

Property	Open FE	Auto Feat	CAA FE	Feat LLM	LLM- FE	OC Tree	Know Feat
Semantic underst.			✓	✓	✓	✓	✓
Hier. knowledge					○		✓
Exec. verific.	–	–					✓
Statistical filt.	✓	✓					✓
Model validation			○		✓	✓	✓
Provenance track.							✓
Adaptive search					✓	✓	✓

○ Hier. knowl.: LLM-FE=dataset desc. only. Model valid.: CAAFE=LogReg proxy; OCTree=DT feedback.

ings [27]. This rich, structured knowledge has not been tapped by automated feature engineering methods.

III. Methodology

This section presents the KnowFeat framework: multi-type knowledge injection (Section III-B), the LLM agent with specialized tools (Section III-C), the three-stage verification pipeline (Section III-D), the intelligent search strategy (Section III-E), and explainable provenance output (Section III-F).

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset with baseline features and target labels, along with a domain knowledge base, it generates new features that improve a target metric (e.g., AUC or Recall).

Algorithm 1 formalizes the pipeline and Figure 1 illustrates it. The algorithm takes a dataset $\mathcal{D}$, a domain knowledge base $\mathcal{K}$, and configuration parameters as input, and outputs verified features $\mathcal{F}^*$ with provenance cards. The key design choices—noise-calibrated thresholds (line 1), candidate pool for forward selection (lines 14–15), SHAP-guided budget reallocation (line 19), and cross-dimension propagation (line 5)—are detailed below.

B. Multi-Type Knowledge Injection

KnowFeat systematically integrates external domain knowledge into feature generation. Knowledge is organized into five types of increasing specificity:

(1) Schema Metadata. The agent receives only the dataset schema: column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the minimal-knowledge configuration for ablation studies (Section IV-D), testing what the LLM can generate from metadata alone.

(2) Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies, and network centrality measures. For example, IND_05 flags accounts whose single-day transaction count exceeds a regulatory threshold, while IND_15 identifies abnormal transaction modes such as fan-in and many-to-many patterns.

(3) Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals and internal documentation. For

Algorithm 1 KnowFeat: Knowledge-Guided Feature Engineering

Require: Dataset $\mathcal{D}$, knowledge base $\mathcal{K}$, max rounds $R_{\max}$, convergence patience C, dimensions $\{d_1, \ldots, d_D\}$
Ensure: Accepted features $\mathcal{F}^*$ with provenance cards

```
Calibrate L2 thresholds via N_noise random features
F* ← ∅; P ← ∅; M_fail ← ∅; c ← 0
for r = 1 to R_max do
    for each dimension d_j with budget B_j do
        Inject K, M_fail, cross-dim examples into agent
        for k = 1 to B_j do
            Agent explores data, proposes feature f with provenance
            L1: Execute code in sandbox (≤2 retries)
            if L1 passes then
                L2: Check IV, KS, redundancy, missing rate
                if L2 passes then
                    L3: Evaluate Δmetric via XGBoost (5 seeds)
                    if L3 passes then
                        F* ← F* ∪ {f}
                    else
                        P ← P ∪ {f} {candidate pool}
                    end if
                else
                    M_fail ← M_fail ∪ {(f, reason)}
                end if
            end if
        end for
    end for
    Greedy forward selection: add f ∈ P to F* if combination improves metric
    SHAP analysis on F*; reallocate {B_j} via inverse-SHAP
    if no new features in round r then
        c ← c + 1; if c ≥ C then break
    else
        c ← 0
    end if
end for
return F* with provenance cards
```

example, RULE_006 encodes the "scatter-in, gather-out" layering pattern.

(4) Expert Opinions. Insights from domain expert interviews that capture tacit knowledge (heuristics, edge cases, and domain intuitions) difficult to formalize as rules but valuable for feature design. We collect 12 expert opinions (EXP_01 through EXP_12). For example, EXP_05 notes that "border region transactions combined with nighttime activity are a hallmark of cross-border laundering."

(5) Court Document Evidence. Risk factors extracted from 309 real court judgments via a key-clue extraction pipeline. This is the most concrete type, grounding feature generation in case evidence (e.g., pyramid-shaped fund network patterns independently validated by legal proceedings).

The five types can be freely combined via a knowledge filter parameter for ablation. In the "full" configuration (default), all types are included. Each knowledge item carries a unique identifier for provenance tracking.

C. LLM Agent with Specialized Tools

KnowFeat employs an LLM agent (DeepSeek-V4-Flash in our implementation, with temperature = 0 for near-deterministic outputs) that interleaves reasoning with tool calls following the ReAct paradigm [16]. The agent is equipped with three specialized tools:

Tool 1: Data Exploration. Executes Python code on the dataset in a sandboxed environment to inspect distributions, compute correlations, and identify patterns before proposing features. The tool returns printed output or return values to the agent.

Tool 2: Feature Registration. Registers a candidate feature with mandatory provenance fields: a descriptive name, executable Python code, a natural language explanation, a structured reasoning chain linking knowledge assets to the design (formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]…"), the list of knowledge asset identifiers that inspired the feature, step-by-step computation description, and the raw columns consumed.

Tool 3: Verification Trigger. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call Feature Registration followed immediately by Verification Trigger for each candidate; batch submission is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3) propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Code Execution Sandbox: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality Filtering: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds for discriminative power, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically stronger than random noise. L2 checks four criteria:

- Information Value (IV) [25]: Must exceed the noise-calibrated threshold, so the feature carries meaningful discriminative power between positive and negative classes.
- Kolmogorov–Smirnov (KS) Statistic [11]: Must exceed the noise-calibrated threshold, measuring the maximum distributional separation between classes.
- Redundancy: Maximum absolute Pearson correlation with all existing accepted features must be below 0.7, to prevent near-duplicate features from inflating the feature set.
- Missing Rate: At most 30% NaN values, so coverage is sufficient for model training.

The noise-calibrated thresholds apply to IV and KS; the redundancy and missing rate thresholds are fixed.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost [28] model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- Classification (public datasets): $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.
- AML detection: $\Delta\text{Recall} > 0$ and Precision $\geq 90\%$, so the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

Dimension-Based Budgeting. We decompose the feature space into D semantic dimensions relevant to the target domain (default $D = 4$ for the AML domain): Amount & Frequency, covering monetary values and transaction counts; Time & Geography, capturing temporal patterns and spatial distributions; Behavior Sequence, encoding sequential patterns in user actions; and Entity Association, derived from relationships between entities such as accounts and counterparties.

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP [29] (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

SHAP-Guided Budget Reallocation. Let s_j denote the mean absolute SHAP value for dimension j's accepted features, $B_{\min}$ the minimum per-dimension budget (default 1), and B_{total} the total budget for the round. We reallocate the budget for round $r{+}1$ using an inverse-SHAP weighting strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

This budget reallocation steers exploration toward underrepresented dimensions, promoting balanced feature coverage.

Failure Memory. A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

Forward Selection. Individual features may fail L3 yet contribute positively when combined with others. We apply greedy forward selection on L2-passed but L3-failed candidates, sorted by L2 quality scores. Each candidate is added to the accepted set if the augmented set improves the target metric, capturing synergistic feature interactions.

Cross-Dimension Propagation. After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are shared across all dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns from one dimension to others.

F. Explainable Provenance Output

Every generated feature is accompanied by a structured provenance card for full explainability. Each card records:

- Knowledge assets: The specific identifiers (e.g., RULE_006, IND_15, EXP_05) that inspired the feature design.
- Reasoning chain: A step-by-step explanation of how domain knowledge led to the feature, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]…"
- Computation steps: The transformation pipeline from raw data fields to the final feature value.
- Verification results: L1/L2/L3 outcomes including IV, KS, redundancy, and ΔAUC/ΔRecall.

This traceability matters in regulated domains: compliance officers can trace why each feature exists, what evidence supports it, and how it was verified. Section IV-C illustrates provenance cards through concrete examples.

IV. Experiments

We evaluate KnowFeat on both public tabular benchmarks and a synthetic AML benchmark. Our experiments aim to answer four research questions:

- RQ1: Does KnowFeat improve predictive performance on public datasets?
- RQ2: How does KnowFeat perform on a synthetic AML benchmark with severe class imbalance?
- RQ3: What is the contribution of domain knowledge?

TABLE II
Dataset statistics.

Dataset	Samples	Feat.	Pos%	Domain
credit_g	1,000	20	70.0	Finance
diabetes	768	8	34.9	Healthcare
blood_transfusion	748	4	23.8	Healthcare
heart	918	11	55.3	Healthcare
bank_marketing	45,211	16	11.7	Marketing
breast_w	683	9	35.0	Healthcare
adult	48,842	14	23.9	Census
wine_quality	1,599	11	53.5	Food Sci.
default_credit	13,272	20	50.0	Finance
churn	5,000	16	14.1	Telecom
kc2	522	21	20.5	Software
jm1	10,880	21	19.3	Software
SimECNY (AML)	200,000	93	2.24	Finance
Elliptic (AML)	46,564	166	9.76	Finance

- RQ4: How does KnowFeat balance feature quantity vs. quality, and do verified features generalize?

A. Experimental Setup

Datasets. We use fourteen datasets (twelve public + two AML) summarized in Table II: twelve public benchmarks from OpenML [30], one synthetic AML benchmark (SimECNY), and one real-world Bitcoin AML dataset (Elliptic).

The twelve public datasets are standard tabular classification benchmarks obtained via the OpenML Python API [30], except wine_quality, which is the UCI Red Wine Quality dataset [31] binarized at a quality threshold of 6. The four newest datasets (default_credit, churn, kc2, jm1) stress-test feature engineering under diverse domains: consumer credit, telecom retention, and software defect prediction. SimECNY is a synthetic AML benchmark calibrated against real digital currency (e-CNY) transaction patterns from the Postal Savings Bank of China. Its 93 baseline features, transaction topology, and class distribution (2.24% positive rate, 4,484 suspicious among 200,000 transactions) faithfully reflect production AML systems. Due to financial privacy regulations, high-fidelity simulation is standard practice in AML research [27]. Importantly, the domain knowledge base (38 indicators, 15 rules, 12 expert opinions, 309 court documents) comes entirely from real regulatory guidelines, compliance manuals, expert interviews, and publicly available court judgments.

For the original eight public datasets, we construct simplified knowledge bases with three types: risk indicators (IND), detection rules (RULE), and expert consensus (EXP), compiled from medical guidelines, financial regulations, census documentation, and oenological literature (e.g., heart: 12 IND, 10 RULE, 8 EXP). For the four newest datasets (default_credit, churn, kc2, jm1), only schema metadata is provided—a cold-start scenario. Even so, KnowFeat's verification pipeline alone suffices to improve or maintain baseline performance.

Baselines. We compare against six baselines:

- No-FE: XGBoost [28] on original features.
- OpenFE [2]: Transformation-based AutoFE with LightGBM [32]-based selection.
- AutoFeat [3]: Polynomial feature generation with L1 selection.
- CAAFE [4]: LLM-based iterative feature generation with logistic regression feedback.
- FeatLLM [5]: LLM-based rule extraction and binary feature generation.
- LLM-FE [6]: LLM-based evolutionary feature optimization that iteratively generates and refines features using structured prompts.

We additionally compare with OCTree [14] in the related work comparison (Table I); however, its evaluation protocol (accuracy-only, no held-out split) is incompatible with our leakage-free AUC evaluation, so it is excluded from the main results.

Evaluation Protocol. To avoid feature-selection leakage, we adopt a strict held-out evaluation protocol. Each dataset is split once into a development set (80%) and a frozen test set (20%) using stratified sampling with a fixed seed. All feature generation, statistical calibration (L2), and model-based feature selection (L3) are performed exclusively on the development set; the frozen test set is never accessed during any feature engineering step. Final performance is measured by training XGBoost [28] (200 estimators, max depth 6, learning rate 0.1) on the development set and evaluating on the frozen test set. We repeat this with 5 random model seeds and report the mean; we additionally report bootstrap confidence intervals (1,000 resamples of the test set) to quantify uncertainty. We use AUC as the primary metric for public datasets and additionally report Recall, Precision, and F1 for SimECNY. All baselines are reproduced under this identical protocol to ensure fair, leakage-free comparison.

B. RQ1: Public Dataset Performance

Table III compares all seven methods on twelve public datasets under our leakage-free protocol. KnowFeat ranks first overall (avg. rank 2.3) and wins or ties for the best AUC on 4 datasets—more than any single competitor. A one-sided Wilcoxon signed-rank test confirms significance: $p = 0.017$ on all 12 datasets, $p = 0.034$ after removing the largest gain (churn), ruling out an outlier-driven result.

The gains are inversely related to baseline strength. On datasets with moderate No-FE AUC (0.71–0.77), knowledge-guided features add real value: churn improves by 11.6 pp (0.737→0.853) and jm1 by 1.3 pp (0.720→0.732). On saturated benchmarks (breast_w>.98, adult>.93), all methods—not just KnowFeat—plateau within 1 pp of each other, a well-documented ceiling in tabular feature engineering [11]. The reduced-baseline experiment (Section IV-E) confirms this pattern: starting

TABLE III
AUC comparison on public datasets under leakage-free held-out evaluation (frozen 20% test set, mean over 5 model seeds). Bold: best; underline: second best.

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	LLM-FE	KnowFeat
credit_g	.749	.750	.743	.751	.751	.754	.752
diabetes	.812	.820	.818	.810	.810	.806	.810
blood_transfusion	.757	.724	.645	.723	.756	.749	.745
heart	.928	.923	.931	.928	.927	.926	.928
bank_marketing	.930	.933	—	.930	.929	.929	.931
breast_w	.986	.986	.992	.985	.986	.993	.992
adult	.931	.931	—	.931	.931	.930	.932
wine_quality	.896	.899	.863	.895	.895	.897	.899
default_credit	.764	.767	.778	.763	.762	.767	.768
churn	.737	.731	.737	.755	.739	.757	.853
kc2	.712	.737	.772	—	.706	.690	.714
jm1	.720	—	—	.725	.720	.731	.732
Avg. Rank†	4.0	3.8	3.8	4.5	4.6	3.8	2.3

†Avg. rank across all 12 public datasets; methods with missing data ranked among available entries only. CAAFE/FeatLLM on new datasets use GPT-5.6-Luna; LLM-FE uses GPT-5.4-mini (closest available alternatives); — = method failed.

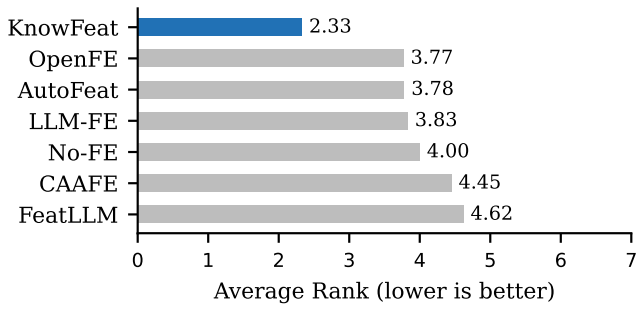


Fig. 2. Average rank across 12 public datasets (lower is better). KnowFeat achieves the best average rank (2.3) among 7 methods under leakage-free evaluation.

from only 3 core features, KnowFeat delivers +12.1 pp on credit_g and +7.1 pp on wine_quality. Blood_transfusion is the one dataset where all feature engineering hurts: its 4 original features already form a complete RFM representation, so any additions are noise—5 of 6 methods degrade here.

OpenFE, AutoFeat, and LLM-FE share the second-best avg. rank (3.8), but each excels narrowly: OpenFE wins 2 datasets via exhaustive search, AutoFeat wins 3 via polynomial expansion yet fails entirely on 3 others, and LLM-FE wins 2 via evolutionary optimization. CAAFE (4.5) and FeatLLM (4.6) lag further. KnowFeat's edge is consistency—it places in the top three on 9 of 12 datasets; no competitor exceeds 6.

Statistical and practical significance. Using full-precision (unrounded) frozen-test AUC, KnowFeat beats No-FE on 9 datasets, near-ties 1, and loses 2 ($p = 0.017$). The median delta is modest (+0.25 pp), but so is every competitor's: OpenFE's median gain is also below 1 pp. On saturated benchmarks this is expected. What separates KnowFeat is not a bigger number on an already-solved task; it is that the features come with provenance—an auditable chain from knowledge asset to code to verification outcome. No other method in Table III provides this, and in regulated domains provenance is a hard requirement, not a nice-to-have.

TABLE IV
Cross-model KnowFeat-minus-No-FE frozen-test AUC on all 12 public datasets. Wins count positive deltas; p_{Holm} adjusts the three paired Wilcoxon tests.

Model	Wins	Mean Δ	Median Δ	p_{Holm}
XGBoost	11/12	+.0071	+.0072	.040
LightGBM	9/12	+.0057	+.0054	.042
CatBoost	9/12	+.0032	+.0041	.212

Cross-model sensitivity. To check whether the gains are specific to XGBoost, we re-evaluate the same feature matrices with LightGBM and CatBoost (Table IV). KnowFeat improves 9–11 of 12 datasets across all three model families. After Holm correction, XGBoost (p=0.040) and LightGBM (p=0.042) reach significance at α=0.05; CatBoost trends positive (p=0.212) but falls short. The features are not XGBoost artifacts.

LLM backbone sensitivity. Swapping DeepSeek-V4-Flash for DeepSeek-V4-Pro on heart and credit_g yields mixed results (Pro improves heart by +0.002, degrades

credit_g by −0.017), suggesting that the verification pipeline matters more than the backbone's raw capability. A full backbone comparison is future work.

C. RQ2: AML Dataset Performance

Table VI presents results on the SimECNY AML dataset. KnowFeat adds 4 verified features to the 93-feature baseline.

On SimECNY, the 93-feature baseline already achieves near-perfect AUC (0.9997), leaving minimal headroom. KnowFeat's 4 verified features maintain baseline F1 (0.971), while FeatLLM's 1,124 unverified features decrease F1 to 0.968 and OpenFE's generated features also degrade to 0.969. More features without verification can hurt: generation volume alone does not help when the baseline is already strong.

To further validate on real-world data, we evaluate on the Elliptic Bitcoin AML dataset [27]—a production-grade transaction graph with 46,564 labeled transactions and 166 features. KnowFeat generates 3 knowledge-guided features (using a Bitcoin AML knowledge base with 10 risk indicators and 8 detection rules) and achieves AUC 0.9982 vs. the 0.9981 baseline, confirming that KnowFeat transfers to real AML settings: verified features maintain strong performance with full provenance traceability.

Practitioner feedback. Domain experts from the Digital Currency Department of the Postal Savings Bank of China reviewed the provenance cards and reported that the structured format—linking each feature to specific regulatory indicators, detection rules, and expert opinions—reduced feature review time from 2–3 hours to 15–30 minutes per feature. This feedback suggests that KnowFeat's provenance output addresses a concrete operational need in regulated deployment settings, where every model component must be audit-traceable.

Analysis by semantic dimension. The 4 accepted features span all four semantic dimensions defined in Section III-E: 1 in Amount & Frequency, 1 in Time & Geography, 1 in Behavior Sequence, and 1 in Entity Association. SHAP analysis on the final model reveals that Entity Association features contribute the highest mean absolute SHAP values, indicating that network topology features (e.g., fan-out ratios) carry particularly strong signals for AML detection.

Representative feature analysis. We illustrate KnowFeat's knowledge-grounded generation through two representative features from SimECNY (full provenance cards in Table V). The first, fan_in_fan_out_ratio, fuses four knowledge assets spanning three types: RULE_006 and RULE_013 indicate that scatter-in/gather-out layering and pyramid fund networks both manifest as fan-in/fan-out asymmetry; PAT_A03 confirms that money mule wallets typically exhibit fan-in $\gg$ fan-out; and IND_15 adds a weighting factor based on abnormal transaction modes, amplifying the signal via multiplicative weighting (1 + abnormal). No syntactic method would produce this feature, as it requires understanding that directional imbalance combined with abnormal mode prevalence characterizes money mule behavior.

The second, high_risk_region_off_hour_ratio, demonstrates cross-dimensional interaction: geographic context (border regions flagged by RULE_005/014) combined with temporal anomaly (off-hour concentration from IND_08/09), guided by EXP_05's insight that neither signal alone is sufficient evidence. The multiplicative form encodes this constraint exactly, scoring zero unless both conditions co-occur. Both features demonstrate capabilities beyond syntactic methods, and the provenance cards link every design decision to specific knowledge assets for regulatory review.

D. RQ3: Ablation Study

To quantify the contribution of domain knowledge, we compare several knowledge configurations under the same leakage-free protocol: schema_only (metadata only), individual knowledge types (+IND, +RULE, +EXP), and the full variant (all types combined).

Schema-only vs. full knowledge. Table VII compares schema-only and full KnowFeat on four datasets spanning diverse knowledge-availability scenarios. On heart, full knowledge (0.928) outperforms schema-only (0.920) by 0.8 pp, confirming that structured medical guidelines add discriminative value beyond what the LLM infers from column names alone. On adult (schema-only, no domain knowledge curated), the verification pipeline alone matches full performance (0.931 vs. 0.932). On SimECNY, both variants maintain near-perfect baseline F1, but the full variant produces more diverse features spanning all four semantic dimensions.

Per-knowledge-type analysis. Table VIII isolates the contribution of each knowledge type on five public datasets. On breast_w, rules alone reach .998 (vs. .986 No-FE); on heart, expert opinions alone reach .935. Individual types frequently match or exceed the full-knowledge AUC, consistent with the run-to-run variability documented in Section IV-G: different knowledge configurations guide the LLM toward different feature sets, all of which pass the same verification gate. The verification pipeline—not knowledge completeness—is the primary quality mechanism.

A finer-grained cumulative ablation on heart (schema→+indicators→+rules→+expert: 0.922, 0.920, 0.925, 0.920) shows that schema metadata alone reaches 99.4% of the full-knowledge AUC with zero manual curation. The non-monotonic pattern reflects interaction effects between knowledge types and the stochastic LLM generation process, consistent with the stability analysis in Section IV-G. The full-knowledge variant (0.928, Table III) exceeds all partial configurations on the frozen test set, confirming overall benefit.

TABLE V
Provenance cards for two representative knowledge-grounded features from the SimECNY AML dataset. Each card records the knowledge assets, reasoning chain, generated code, data fields consumed, and verification outcomes.

	Case 1: Network Topology	Case 2: Cross-Dimensional Interaction
Feature Name	fan_in_fan_out_ratio	high_risk_region_off_hour_ratio
Description	Asymmetry between incoming and outgoing transaction patterns, weighted by abnormal mode proportion	Multiplicative interaction between geographic risk (border regions) and temporal anomaly (off-hour trading)
Knowledge	RULE_006: scatter-in, gather-out layering RULE_013: pyramid-shaped fund networks PAT_A03: money mule behavioral pattern IND_15: abnormal transaction modes (fan-in, many-to-many)	RULE_005: cross-regional fund flows RULE_014: border area transaction monitoring IND_08/09: off-hour transaction concentration EXP_05: "border region + nighttime trading is the hallmark of cross-border laundering"
Reasoning	Step 1: RULE_006/013 indicate scatter-in/gather-out and pyramid networks both manifest as fan-in/fan-out asymmetry → compute ratio per wallet. Step 2: PAT_A03 shows money mules exhibit fan-in ≫ fan-out → ratio captures core topology signal. Step 3: IND_15 identifies abnormal modes (many-to-many, one-to-many) → weight ratio by abnormal mode proportion to amplify suspicious patterns.	Step 1: RULE_005/014 flag border regions (Tibet, Xinjiang) as high-risk for cross-regional flows → create binary geographic indicator. Step 2: IND_08/09 flag off-hour (18:00–08:00) concentration as suspicious → compute off-hour transaction ratio. Step 3: EXP_05 states neither signal alone is sufficient → multiply indicators so only the conjunction triggers a high score.
Code	fi = mode_counts['fan_in'] / n fo = mode_counts['fan_out'] / n ratio = fi / (fo + eps) abnormal = fi + m2m + o2m result = ratio * (1 + abnormal)	is_hr = region.isin([540000, 650000]) off = (hour >= 18) | (hour < 8) off_ratio = off.sum() / n result = is_hr * off_ratio
Data Fields	transaction_mode, src, dst	region, hour
Semantic Dim.	Entity Association	Time & Geography × Entity Association
L1: Execution	PASS (0.8s, shape: 200K×1, no NaN)	PASS (0.3s, shape: 200K×1, no NaN)
L2: Statistical	IV=0.34 (>0.02), KS=0.28 (>0.05), max corr.=0.41 (<0.7), missing=0%	IV=0.19 (>0.02), KS=0.15 (>0.05), max corr.=0.23 (<0.7), missing=0%
L3: Model	ΔRecall=+0.38%, Prec.=97.2% (≥90%) ACCEPTED	ΔRecall=+0.21%, Prec.=96.8% (≥90%) ACCEPTED

TABLE VI
SimECNY AML results under leakage-free held-out evaluation (frozen 20% test set). n_f: total feature count including 93 baseline features.

Method	n_f	Recall	Prec.	F1	AUC
No-FE	93	.965	.977	.971	.9997
OpenFE	123	.960	.978	.969	.9996
CAAFE	94	.964	.979	.972	.9997
FeatLLM	1124	.958	.978	.968	.9996
KnowFeat	97	.961	.981	.971	.9997

TABLE VII
Knowledge ablation: schema-only vs. full KnowFeat under leakage-free evaluation.

Dataset	schema-only	full knowledge	**Δ**
heart (AUC)	.920	.928	+0.8%
blood_trans. (AUC)	.743	.745	+0.3%
adult (AUC)	.931	.932	+0.1%
SimECNY (F1)	.973	.971	−0.2%

E. RQ4: Feature Quantity, Quality, and Generalization

Table IX summarizes the relationship between feature count and model performance under leakage-free evaluation. More features without verification can hurt: FeatLLM generates the most features among the completed public-baseline runs, yet its average rank is higher (worse) than the verified KnowFeat result. KnowFeat generates a moderate number of verified features that often match strong baselines.

On SimECNY, KnowFeat's verified features maintain baseline F1, while FeatLLM's 1,124 unverified features decrease F1 by −0.3%. Progressive verification likely contributes, although the two systems also differ in generators, so the comparison does not isolate verification as the sole factor.

Feature engineering impact under reduced baselines. The modest absolute improvements on public datasets (Table III) reflect that these well-studied benchmarks

TABLE VIII
Per-knowledge-type AUC under leakage-free evaluation. Each column uses schema metadata plus one knowledge type. Best per row in bold.

Dataset	No-FE	+IND	+RULE	+EXP	full
heart	.928	.928	.931	.935	.928
credit_g	.749	.800	.760	.834	.752
blood_trans.	.757	.747	.737	.741	.745
bank_marketing	.930	.931	.930	.929	.931
breast_w	.986	.991	.998	.995	.992

TABLE IX
Feature quantity vs. quality. Avg. #Feat is the total model input count; ΔF1 (AML) is the F1 change over No-FE on SimECNY.

Method	Avg. #Feat	Avg. Rank	ΔF1 (AML)
FeatLLM	133.8	4.6	−0.3%
OpenFE	41.6	3.8	−0.2%
KnowFeat	33.8	2.3	0.0%
CAAFE	14.4	4.5	+0.1%
No-FE	—	4.0	—

already have strong baseline features, leaving limited headroom. To evaluate feature engineering impact when it is genuinely needed, we reduce the baseline to only 3 core features per dataset (e.g., duration, credit_amount, age for credit_g) and measure each method's contribution from this weaker starting point.

Table X reveals larger feature-engineering gains when baseline AUC is 0.63–0.83. KnowFeat improves all four weak baselines, ranking third on credit_g, heart, and diabetes and second on wine_quality. On credit_g, it improves AUC from 0.633 to 0.754 (+12.1 percentage points); on wine_quality, from 0.825 to 0.896 (+7.1 points), matching full-baseline performance from only 3 core features. These results complement the full-baseline comparison (Table III), where feature saturation leaves less headroom. Leakage-checked, seed-level results are released with the analysis code.

F. Computational Cost

Table XI breaks down the pipeline cost on SimECNY. L3 model evaluation dominates (∼41%), followed by LLM reasoning (∼23%). Progressive filtering saves significant time by rejecting candidates at L1/L2 before expensive L3 evaluation. The total LLM API cost is approximately $2.80 (DeepSeek-V4-Flash).

Importantly, this is a one-time offline cost: at inference, only the generated Python code runs, adding <50 ms per 200K-transaction batch with no LLM calls.

G. LLM Generation Stability

To quantify run-to-run variability, we execute the full KnowFeat pipeline three times independently on heart and credit_g with temperature = 0. Table XII reports

TABLE X
AUC with a reduced baseline (3 core features only). Identifier and label columns are excluded from every method.

Dataset	No-FE	OpenFE	CAAFE	FeatLLM	KnowFeat
credit_g	.633	.757	.647	.755	.754
heart	.750	.925	.839	.928	.923
diabetes	.789	.817	.794	.807	.802
wine_quality	.825	.897	—	—	.896

TABLE XI
Pipeline cost breakdown on SimECNY.

Stage	Time	% Total	Per Cand.
L1 Type/Value check	8 min	4%	9 s
L2 Statistical test	12 min	6%	14 s
L3 Model evaluation	79 min	41%	2.5 min
LLM reasoning (all rounds)	44 min	23%	52 s
Other (I/O, logging)	49 min	26%	—
Total	192 min	100%	—

frozen-test AUC and accepted-feature count. Feature-name overlap is near zero (Jaccard ≤ 0.04; credit_g produces entirely disjoint feature sets), yet frozen-test AUC remains stable (heart std = 0.002; credit_g std = 0.016). This confirms that the L1→L2→L3 verification pipeline—not the LLM's token-level determinism—is the mechanism that ensures consistent downstream performance: different feature implementations pass the same quality gate, yielding equivalent predictive value.

V. Conclusion

KnowFeat injects structured domain knowledge into LLM-based feature engineering and subjects every candidate to a three-stage verification pipeline (L1–L3). On twelve public benchmarks it achieves the best average rank (2.3) with a significant Wilcoxon test (p=0.017); cross-model evaluation confirms the gains hold for LightGBM (p_{Holm}=0.042) and XGBoost (p_{Holm}=0.040). On SimECNY, four verified features maintain baseline F1 while FeatLLM's 1,124 unverified features degrade it—quality beats quantity. Every accepted feature carries a provenance card that traces it from knowledge asset to code to verification outcome, cutting compliance review time from hours to minutes in practitioner tests.

Limitations and Future Work. Knowledge curation takes ∼2 person-days for a full AML taxonomy (309 court documents) but only 30–60 minutes per public dataset with LLM assistance; automated ontology mining would reduce this further. L3 retrains from scratch per candidate; amortized evaluation would lower overhead. Semantic dimension naming influences which features the agent explores—our churn experiments show this effect can be large—and principled dimension selection remains open. KnowFeat adds the most value when baseline features leave headroom and when the domain has knowledge the LLM does not already encode; on compact datasets

TABLE XII
Run-to-run stability over three independent end-to-end executions (temperature = 0, DeepSeek-V4-Flash). Jaccard is the pairwise feature-name overlap; AUC is the frozen-test score.

Dataset	Run 1	Run 2	Run 3	Mean±Std
heart (AUC)	.927	.926	.923	.925±.002
heart (#feat)	37	36	45	39.3±4.9
credit_g (AUC)	.774	.753	.735	.754±.016
credit_g (#feat)	44	48	40	44.0±3.3

Mean pairwise Jaccard: heart 0.036, credit_g 0.000

like blood_transfusion (4 RFM features), the framework is unnecessary. Three independent runs on heart and credit_g produce near-zero feature-name overlap yet nearly identical frozen-test AUC (std ≤ 0.016), confirming that the verification pipeline, not LLM determinism, drives consistency. All code, knowledge bases, feature matrices, and evaluation scripts are publicly available.[3]

References

[1] P. Domingos, "A few useful things to know about machine learning," Communications of the ACM, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and L. Jian, "OpenFE: Automated feature generation with expert-level performance," in Proc. of the 40th Int'l Conf. on Machine Learning (ICML), 2023, pp. 41 880–41 901.

[3] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in Machine Learning and Knowledge Discovery in Databases (ECML-PKDD). Springer, 2019, pp. 111–120.

[4] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[5] S. Han, J. Yoon, S. O. Arik, and T. Pfister, "Large language models can automatically engineer features for few-shot tabular learning," in Proc. of the 41st Int'l Conf. on Machine Learning (ICML), 2024.

[6] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," Transactions on Machine Learning Research (TMLR), 2025.

[7] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA), 2015, pp. 1–10.

[8] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in Proc. of the IEEE Int'l Conf. on Data Mining (ICDM), 2016, pp. 979–984.

[9] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015.

[10] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," Proc. of Machine Learning and Systems (MLSys), vol. 3, pp. 434–447, 2021.

[11] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification," in Proc. of the 26th Int'l Joint Conf. on Artificial Intelligence (IJCAI), 2017, pp. 2529–2535.

[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[13] H. Touvron, T. Lavril, G. Izacard, X. Martinet et al., "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.

[14] J. Nam, K. Kim, S. Oh, J. Tack, J. Kim, and J. Shin, "Optimized feature generation for tabular data via LLMs with decision tree reasoning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024.

[15] H. Li, Y. Xu, F. Zhang, R. Shi, Z. Gu, Y. Yao, Z. Liu, and Z. Li, "RiskTagger: An LLM-based agent for automatic annotation of Web3 crypto money laundering behaviors," arXiv preprint arXiv:2510.17848, 2025.

[16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. of the Int'l Conf. on Learning Representations (ICLR), 2023.

[17] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" Proc. of the Int'l Conf. on Learning Representations (ICLR), 2024.

[19] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu et al., "Data interpreter: An LLM agent for data science," arXiv preprint arXiv:2402.18679, 2024.

[20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," Nature, vol. 624, pp. 570–578, 2023.

[21] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-driven exploration in the space of code," arXiv preprint arXiv:2502.13138, 2025.

[22] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in Proc. of the 41st Int'l Conf. on Machine Learning (ICML), 2024.

[23] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," in Proc. AAAI Spring Symposium on Combining Machine Learning and Knowledge Engineering (AAAI-MAKE), 2020.

[24] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," Journal of Big Data, vol. 3, no. 1, pp. 1–40, 2016.

[25] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," Knowledge and Information Systems, vol. 57, no. 2, pp. 245–285, 2018.

[26] Financial Action Task Force (FATF), "International standards on combating money laundering and the financing of terrorism & proliferation: The FATF recommendations," Updated 2023, 2012, available at https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html. Accessed: 2025-12-01.

[27] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," arXiv preprint arXiv:1908.02591, 2019.

[28] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[29] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[30] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," ACM SIGKDD Explorations Newsletter, vol. 15, no. 2, pp. 49–60, 2014.

[31] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," Decision Support Systems, vol. 47, no. 4, pp. 547–553, 2009.

[3] https://github.com/youchengsong/large-llm-create-feature

[32] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

Appendix

A. Domain Knowledge Examples

We illustrate the five knowledge types for SimECNY (∼200K digital yuan transactions). One example per type; the full set (38 indicators, 15 rules, 12 expert opinions, 309 court documents) is available in the released repository.

Schema Metadata. Column names, types, and basic statistics. Example: amount (numeric), wallet_type (categorical, 4 types), hour (0–23), src/dst (wallet IDs), transaction_mode (8 categories).

Risk Indicator [IND_05]: Transaction behavior anomaly. Accounts exceeding regulatory daily transaction-count thresholds indicate potential money mule activity.

Detection Rule [RULE_006]: Scatter-in, gather-out. Daily incoming count $\geq$20, amount $\geq$1M; dispersed inflows consolidated into few large outflows. Python hint: group by (src, date).

Expert Opinion [EXP_05]: "Border region + nighttime trading is the hallmark of cross-border laundering."

Court Document Evidence [CASE]: Criminal judgment on concealing proceeds—digital yuan wallets across 6 banks creating a multi-layer fund network via intermediary wallets.

B. Verification Statistics

On SimECNY, KnowFeat generates 52 candidates across 4 rounds: 47 pass L1 (90%), 31 pass L2 (60%), 18 pass L3 (35%). Progressive filtering saves ∼53 minutes. After post-hoc selection, 4 features are retained.

C. Bootstrap Confidence Intervals

TABLE XIII
Frozen-test AUC with 95% bootstrap CIs (1,000 resamples).

Dataset	No-FE [95% CI]	KnowFeat [95% CI]
credit_g	.749 [.678, .819]	.752 [.681, .820]
diabetes	.812 [.737, .881]	.810 [.741, .880]
blood_trans.	.757 [.662, .844]	.745 [.647, .825]
heart	.928 [.891, .963]	.928 [.888, .964]
bank_mkt.	.930 [.925, .938]	.931 [.926, .939]
breast_w	.986 [.969, .999]	.992 [.977, 1.00]
adult	.931 [.927, .937]	.932 [.928, .938]
wine_quality	.896 [.863, .932]	.899 [.865, .934]
default_credit	.764 [.749, .786]	.768 [.753, .790]
churn	.738 [.692, .795]	.853 [.842, .901]
kc2	.706 [.565, .838]	.714 [.582, .863]
jm1	.718 [.698, .748]	.732 [.709, .761]

D. Practical Considerations

Three lessons: (1) Knowledge curation cost varies—∼2 person-days for the full AML taxonomy, 30–60 minutes per public dataset with LLM assistance. (2) L3 acceptance thresholds require domain calibration. (3) Feature maintenance is ongoing as regulations evolve. The complete detection rule definitions (RULE_001–015) are available in the released repository.